

【令和最新版】 開発者フレンドリーな HTTP SDK作るには

Creating Developer-Friendly HTTP SDKs



pixiv Inc.
USAMI Kenta

pixiv

お前誰よ



- うさみけんた (@tadsan) / Zonu.EXE / にゃんだーすわん
- ピクシブ株式会社 pixiv事業本部 エンジニア
 - 最近はピクシブ百科事典(dic.pixiv.net)を開発しています
- Emacs Lisper, PHPer
 - Emacs PHP Modeを開発しています(2017年-)
- プログラミング言語にちょっとこだわりのある素人

さて

内容を詰め込んでるので
メモとかは無理です
後日詳細なフォロー
アップを公開します

今回のお題

PHPカンファレンス福岡2023

採択 2023/06/24 10:30～ Fusicホール レギュラートーク(15分)

【令和最新版】開発者フレンドリーなHTTP SDK作るには



うさみけんた  tadsan

☆ 8

あなたのアプリケーションでは外部APIを参照していますでしょうか。

あるいは、あなたの作ったAPIを開発者に利用してほしいと考えていますでしょうか。

外部システムをプログラミング言語から使いやすくするパッケージはSDK(software development kit)と呼ばれることもあり、HTTPリクエストの発行やPHPクラスへのマッピングが主な責務になります。

基本的にはマニュアル通りの設定をしてメソッドを呼び出して使えるようにれば最低限使えるSDKができますが、開発やテストのしやすさという観点ではそれだけでは不足です。

このトークでは開発者とSDKユーザーの立場から、2023年現在で考えられるSDKの理想形とテスト手法について手短かに紹介します。

キーワード

- PSR-7, PSR-18, Asynchronous request
- 🧑 不用意にGuzzleに依存しない
- HTTPPlug Discovery

つまり…？

Model: GPT-4

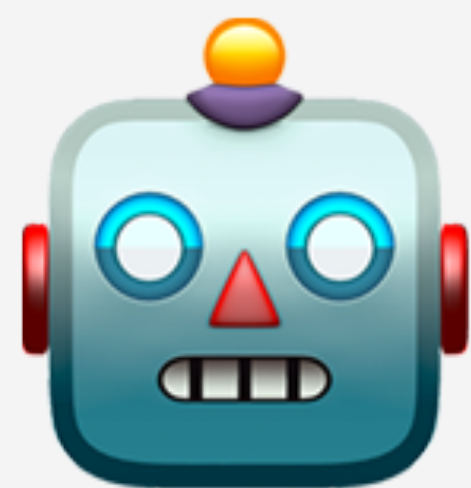


以下の提案を280文字程度に要約してください
(省略)

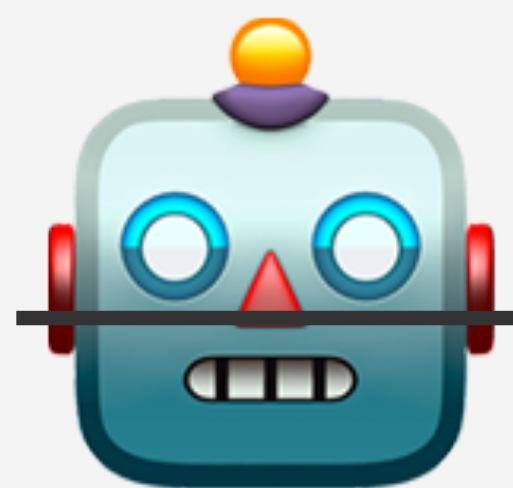


この提案は、アプリケーションの外部APIの参照や自身のAPIを開発者に利用してもらうためのSDKの重要性について述べています。SDKは外部システムとの接続を容易にするだけでなく、開発とテストの易しさも求められます。提案者は、2023年現在の理想的なSDK形状とテスト方法について、開発者とユーザーの視点から簡潔に紹介する予定です。





そういう話です



そういう話です

テストの話はしますが
主題には
なりませんでした

SDKを書く前に おさえておきたい PHPでのHTTPの 使いかた

大昔に別の
アプローチについて
書いたこともあります

WEB+DB PRESS Vol.96(2016年)

第13回

PHP大規模開発入門

PHPからのHTTPリクエスト ファイル操作関数、curl関数、Guzzleライブラリ

ピクシブ(株)

うさみ けんた USAMI Kenta

[mail](mailto:tadsan@pixiv.com) tadsan@pixiv.com [GitHub](https://github.com/zonuexe) zonuexe [Twitter](https://twitter.com/zonu_exe) @zonu_exe

PHPはWebと親和性の高いプログラミング環境です。HTTP(*Hypertext Transfer Protocol*)はWebの標準的な通信プロトコルであり、Webアプリケーションを実装するということは、利用者からのHTTPリクエストを受け付けてHTTPの仕様に沿った応答を返すプログラムを書くことにほかなりません。

実際にはWebサーバやPHP、あるいはWebアプリケーションフレームワークと呼ばれる層が巧妙に隠蔽してくれるので、HTTPについてあまり意識しなくともWebサービスを構築することが可能ですが、今回はPHPから「Webサービスを利用する」という観点でHTTPについて解説します^{注1}。

HTTPの基礎

ます。最も目にする機会が多いのは、<http://wdpress.gihyo.jp/>のようなURL^{注4}の一部分でしょう。このようなURLをブラウザのアドレスバーに入力することでWebサイトを閲覧できます。

Webサイトにも多様な形態があります。たとえばイラスト(画像)を投稿したりほかの利用者が投稿したイラストを閲覧できるpixiv、動画を投稿またはライブ配信したり、それを見ることができるYouTubeなどです。これらのサービスは不特定多数の利用者にアクセスされることを想定していますが、そうではない領域にもHTTPは利用されています。たとえば全文検索エンジンのApache SolrはApache LuceneというJavaのライブラリをHTTPを介して操作できるようにしたフロントエンドです。

LuceneはJavaのライブラリなので、当然Java(も

関数ベースの処理は
イケてないから
Guzzle使おうぜ！
…という内容

Zennに再掲しました(2016年版)





投稿する



PHPからのHTTPリクエスト (2016年版)

php PHP

psr

psr7

 Tech

これは2016年にWEB+DB PRESS Vol.92掲載のPHP大規模開発入門連載『PHPからのHTTPリクエスト ファイル操作関数、curl関数、Guzzleライブラリ』として寄稿した原稿を改稿したものです。

ベースは2016年に書かれた記事ですが、PHPからHTTPリクエストを発行するための基礎について押さえられる内容が含まれており、2023年6月にPHPカンファレンス福岡2023で発表予定の『【令和最新版】開発者フレンドリーなHTTP SDK作るには』の前提となる議論も含めるため、この機

記事を編集する

 著者

 にゃんだーすわん

 公開

2023/06/11

 本文更新

2023/06/13

 文章量

約20,500字

今回しない話（発表時間15分なので）

- OpenAPI(Swagger)の定義からコードを自動生成するとか
- 具体的なSDKの実装方法
 - もうちょっと手前のHTTPの話が中心です
 - 後日、私がいままで作ったSDKをいくつか紹介します
- 具体的なテスト手法について… 間に合いませんでした！
 - スライドじゃなくていい感じにするライブラリをちょこちょこ書いてます

超ざっくり PHP vs HTTP

PHPからHTTPリクエスト

- グローバル関数を使うと超かんたんにHTTPリクエストを発行できる
 - `file_get_contents($url)`
 - 名前に反してPOSTなど任意のメソッドのリクエストを発行できる(!)
 - cURL関数 [`curl_init()`, `curl_exec()`, ...]
- こいつらは事実上の標準関数みたいな感じで外部ライブラリも要らないし簡単だけどリクエスト・レスポンスの組み立てがちょっとめんどい

そうだ、HTTPクライアントを使おう

- 「リクエスト関数」から「クライアントオブジェクト」へ
- PHPではグローバル関数をモックすることはちょっと大変
- インスタンスを外部から受け取ることで、HTTP通信が置換可能になる
- よく知られた実装: ^{ガズル}Guzzle, ^{シンフォニー}Symfony HTTP Client

実際にはuopzで可能
(例: clock-mock)

Guzzle



guzzle

Guzzle, PHP HTTP client

release v7.7.0

ci build <https://github.com/badges/shields/issues/8671>

downloads 535M

Guzzle is a PHP HTTP client that makes it easy to send HTTP requests and trivial to integrate with web services.

- Simple interface for building query strings, POST requests, streaming large uploads, streaming large downloads, using HTTP cookies, uploading JSON data, etc...
- Can send both synchronous and asynchronous requests using the same interface.
- Uses PSR-7 interfaces for requests, responses, and streams. This allows you to utilize other PSR-7 compatible libraries with Guzzle.
- Supports PSR-18 allowing interoperability between other PSR-18 HTTP Clients.
- Abstracts away the underlying HTTP transport, allowing you to write environment and transport agnostic code; i.e., no hard dependency on cURL, PHP streams, sockets, or non-blocking event loops.
- Middleware system allows you to augment and compose client behavior.

つまり…？

Model: GPT-4



日本語280文字で要約



GuzzleはPHPのHTTPクライアントで、HTTPリクエスト送信やWebサービス統合を容易にする。クエリ作成、大量データのストリーミング、JSONデータアップロードなどが可能。同一インターフェースで同期/非同期リクエストを送れる。**PSR-7とPSR-18**に対応、他の互換ライブラリと共有可能。HTTPトランスポートを抽象化し、環境に依存しないコード記述が可能。ミドルウェアシステムでクライアント行動を拡張、構成できる。



PSRとは何か

PHP-FIGとかいう
有志の団体が勧告して
いる標準仕様

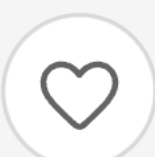
Framework Interop Group

フレームワークを
作ってる人たちの
寄合所帯

PSRの誤解

Qiita

🔍 記事を検索



100



80



 この記事は最終更新日から5年以上が経過しています。



@tadsan (園島 めめ)

投稿日 2018年03月14日 更新日 2018年03月15日

PSRの誤解

 PHP, PSR

先日の[PHPerKaigi 2018](#)はPHPに興味がある多くのひとびとと直接コミュニケーションがとれる貴重な機会でした¹。その中でPSRシリーズへの誤解が聞かれたので一度整理します。

最初にまとめ

ワシのPSRは
20式までであるぞ

一般的にはオートロードや
コーディング規約が知られて
いると思う (たぶん)

PSRではHTTPに
関連する仕様も多く
勧告されている

最重要資料



PHP-FIGのHTTP処理標準の設計はなぜPSR-7/15/17になったのか



PHP-FIGのHTTP
処理標準の設計は
なぜPSR-7/15/
17になったのか



PSR-18は HTTP Clientの インターフェイス

PSR-18は
PSR-15と似ているが
よりシンプル

PSR-15 vs PSR-18

```
<?php

namespace Psr\Http\Server;

use Psr\Http\Message\ResponseInterface;
use Psr\Http\Message\ServerRequestInterface;

interface RequestHandlerInterface
{
    public function handle(ServerRequestInterface $request): ResponseInterface;
}
```

```
<?php

namespace Psr\Http\Client;

use Psr\Http\Message\RequestInterface;
use Psr\Http\Message\ResponseInterface;

interface ClientInterface
{
    public function sendRequest(RequestInterface $request): ResponseInterface;
}
```

人人人人人人人人人人人人人人人人
≥ 僕の知ってるHTTPClientと違う ≤
Y^Y^Y^Y^Y^Y^Y^Y^Y^Y^Y^Y^Y^

僕の知ってる
HTTP Clientって？

GuzzleのREADMEに書いてる

```
$client = new \GuzzleHttp\Client();  
$response = $client->request('GET', 'https://api.github.com/repos/guzzle/guzzle');
```

```
$client->request('GET', 'http://httpbin.org', [  
    'query' => ['foo' => 'bar']  
]);
```

```
$response = $client->get('http://httpbin.org/get');  
$response = $client->delete('http://httpbin.org/delete');  
$response = $client->head('http://httpbin.org/get');  
$response = $client->options('http://httpbin.org/get');  
$response = $client->patch('http://httpbin.org/patch');  
$response = $client->post('http://httpbin.org/post');  
$response = $client->put('http://httpbin.org/put');
```


それに引き換えなんだこのPSR-18

シンプルすぎる

```
<?php
namespace Psr\Http\Client;

use Psr\Http\Message\RequestInterface;
use Psr\Http\Message\ResponseInterface;

interface ClientInterface
{
    public function sendRequest(RequestInterface $request): ResponseInterface;
}
```

だがそれがいい

PSR-7/18 vs Guzzle (1)

- PSRはフレームワークの相互運用できる共通部品となるべく設計された
 - PSR-7/18はHTTPの仕様を(PHPで扱える範囲で)そのまま表現できる
- Guzzleはエンドユーザーが使いやすいように設計されていて、ユーザーはHTTPやPSRの仕様を考えなくても簡単に使えるようになってる
 - リクエスト生成やHTTPヘッダの組み立てなどの実装も含んでいる

PSR-7/18 vs Guzzle (2)

- PSR-7/18は仕様がコンパクトなので気のきいた機能は定義されていない
 - HTTPを理解していないと正しくリクエストを送れないこともありうる
- Guzzleは充実した便利機能を内包しているので、少ない記述量で複数のHTTPリクエストを発行できる
 - なおかつPSR-18のClientInterfaceも実装している
 - Guzzle独自の便利メソッドとPSR-18のシンプルメソッド両方使える

PSR-15 vs PSR-18

```
<?php

namespace Psr\Http\Server;

use Psr\Http\Message\ResponseInterface;
use Psr\Http\Message\ServerRequestInterface;

interface RequestHandlerInterface
{
    public function handle(ServerRequestInterface $request): ResponseInterface;
}
```

```
<?php

namespace Psr\Http\Client;

use Psr\Http\Message\RequestInterface;
use Psr\Http\Message\ResponseInterface;

interface ClientInterface
{
    public function sendRequest(RequestInterface $request): ResponseInterface;
}
```

一年前に話した

PSR-7とPSR-15による Webアプリケーション実装パターン

Web application implementation pattern based on PSR-7 and 15



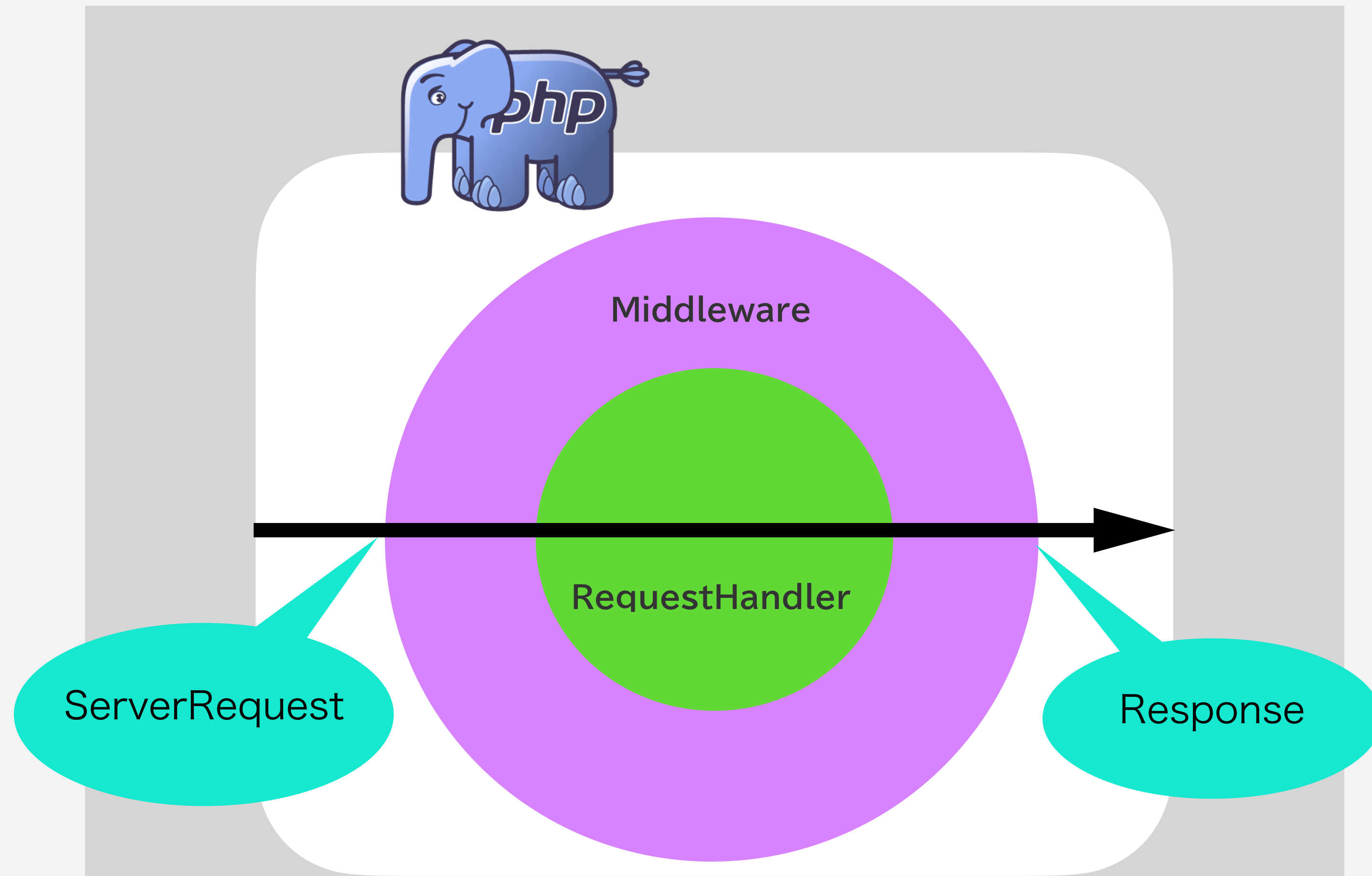
pixiv Inc.
USAMI Kenta

pixiv

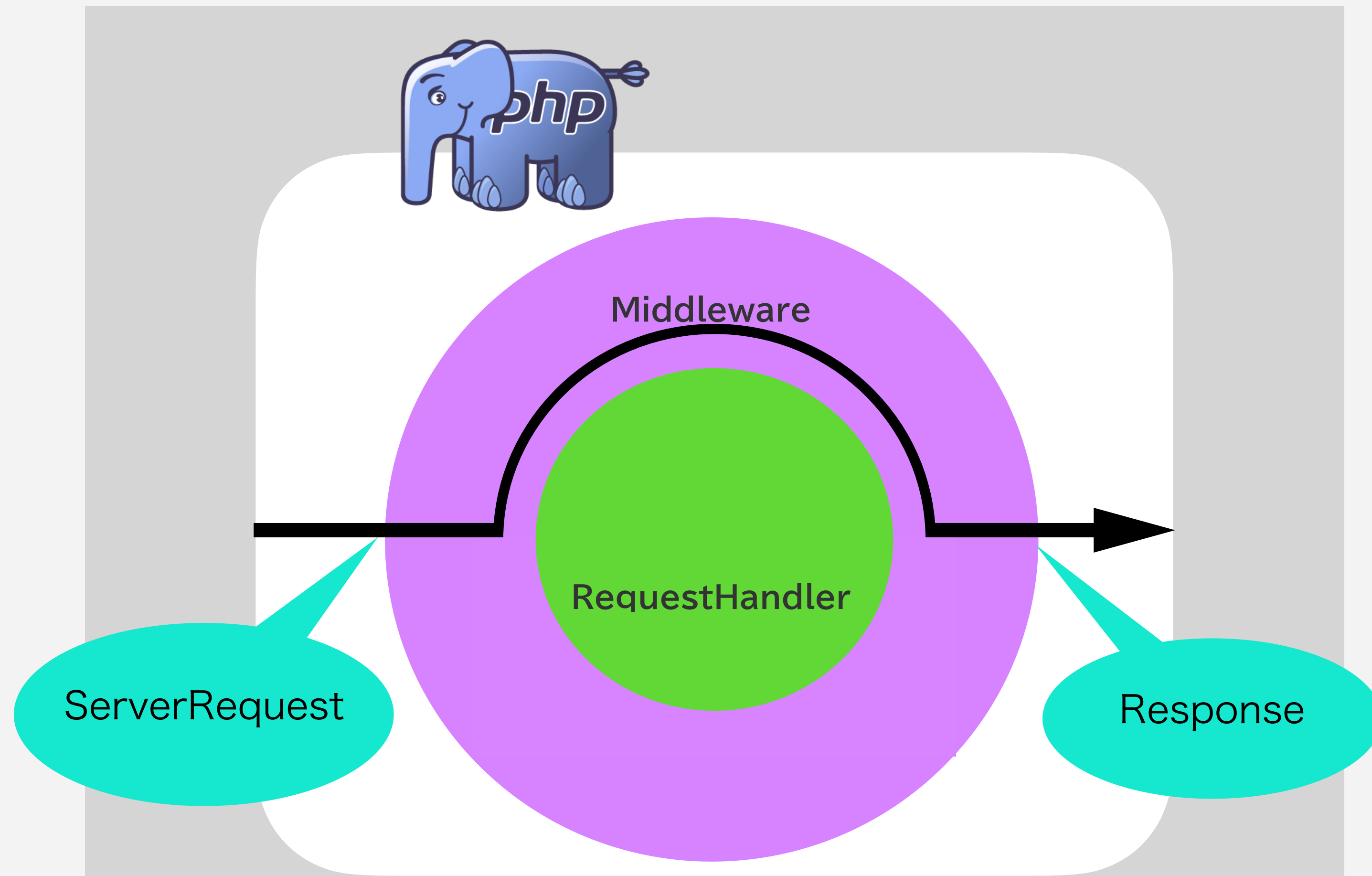
2022-04-11
PHPerKaigi Day 2

pixiv

Middleware呼び出しイメージ



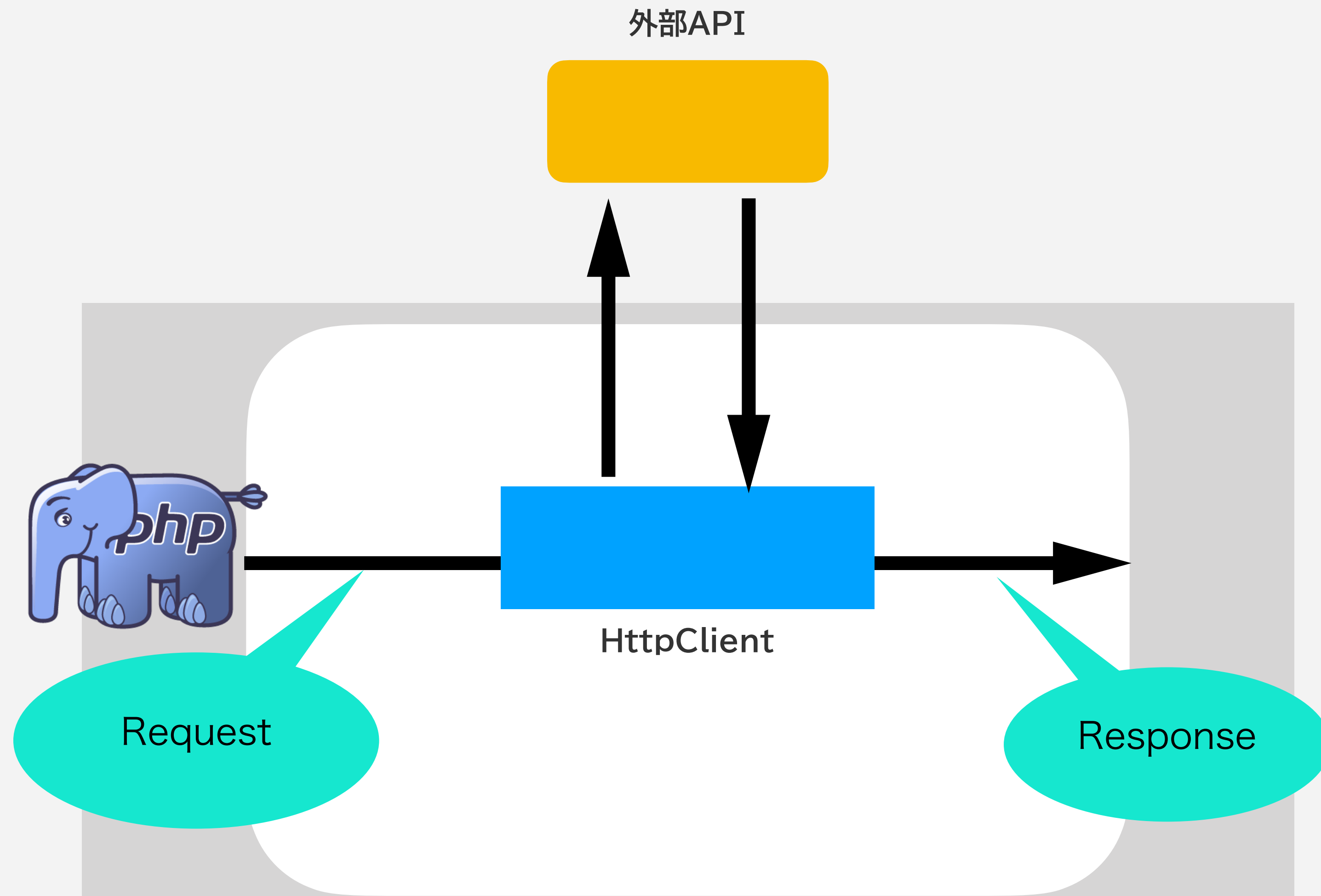
Middleware呼び出しイメージ



特定の場合に
処理を横取りできると
うれしいことがある

例：ユニットテスト

HttpClient呼び出しイメージ



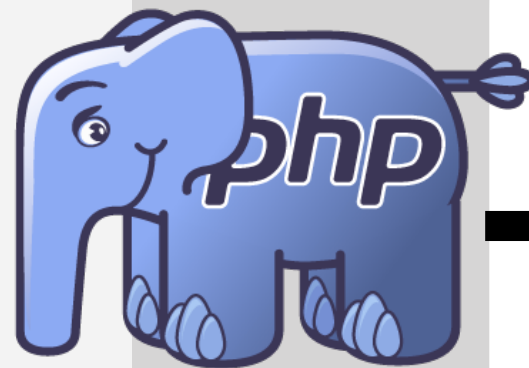
外部にリクエストを送らない

外部API



リクエスト送らない

PHPUnit



Request

MockHttpClient

Response

外部にリクエストが送られないように

- PSR-18の場合、仕様がとてもシンプルなのでClientInterfaceを実装したシンプルな無名クラスを作ってしまうのが簡単
- Guzzleの場合、MockHandlerというハンドラを有効化してテスト用に任意のレスポンスを返す関数を登録してやることができる
- 重要: アプリケーション内に不用意に `new Guzzle\Client()` とか 書かない
 - 普通の方法で処理を横取りできなくなってしまう

依存の注入 (Dependency Injection)

- オブジェクトはメソッド内でその都度生成するのではなく、呼び出し側(もしくはさらに上位のモジュール)から入れ込むことで共通のインターフェイスを持ったものに交換できることを保証する
- 同一機能を持った別のライブラリに乗り換えることもできるし、テスト時に決まった値を返すモックオブジェクトに差し替えることも簡単
- くわしくは「ちよぜつソフトウェア設計入門」を読んで！

ここまでで
準備ができた

【令和最新版】

開発者フレンドリーな

HTTP SDK 作るには

概ねいままで
言ってきたことをやるな
という話

Guzzle便利なのはいいんだけど

- 世の中、Guzzleを使いたいひとばかりではない
 - 僕はkriswallsmith/Buzzという素朴なPSR-18実装が好きです
- SDKとしてはpsr/http-clientとpsr/http-factoryにだけ依存するのが理想
- SDKの実装としてはGuzzleの便利機能に頼れないのを不便に感じることはあるかもしれない

ユーザーの利便性 vs 依存の注入

- ユーザーがSDKを使いはじめるときに事前準備が多すぎると辟易させる
- プログラミングがあんまり得意じゃないユーザーにも使ってもらうことを考えると、事前準備は少なければ少ないほどいい
- 一方で、依存を注入してテストで差し替えたいという利用者の邪魔をしてはいけない

利用者の邪魔をするパターン

- SDKの実装の中で `new Guzzle\Client()` って書く
- SDKの実装の中で `file_get_contents()` とか `curl` 関数を呼ぶ
- ただしこれらのパターンはPHP-VCRというテスト用ライブラリを活用することで通信をフックして任意のレスポンスに書き換えたりすることもできなくはない

php-http/discovery

- php-httpという団体が提供しているライブラリ
 - 動作実績のとれているHTTPライブラリがハードコードされている
- インストール済みのPSR-HTTPライブラリを検出して自動で生成してくれる
 - これはstaticに状態を持っておりDIパターンではないが、DIが使えない環境における強い味方
- 欠点:ドキュメントが不親切め

ユーザーを邪魔しない
使いやすいSDKを
作ろう

【令和最新版】

開発者フレンドリーな

HTTP SDK 作るには